DIAMOND BOOKING

INDEPENDENT AUDIT — FINDINGS & REMEDIATION

October 19, 2026 · Full first-party review of the platform working tree, audited at commit 1d108d5 and remediated through 5165991 — cross-referenced against the Consolidated Master Audit of 18 October. A43–A45 were found during staging preparation.

4	18	18	5
CRITICAL	HIGH	MEDIUM	LOW

39	5	1
FIXED	PARTIAL	WITHDRAWN

What this document is

An independent, line-by-line audit of the code in C:\Users\USER\Desktop\diamond-booking\platform at commit 1d108d5. Every finding below was read in the source, not inferred from documentation. Where a finding confirms, sharpens, corrects or downgrades an item in the attached Consolidated Master Audit (F01–F16 / D01–D12 / S01–S05), the mapping is stated on the finding.

Executive decision

NOT READY TO LAUNCH AS AUDITED. Four CRITICAL defects are confirmed in the source, and the first of them — any signed-in user can promote themselves to platform super-admin with a single table update — is a total compromise of every tenant's data, reachable from a browser with the public anon key. Alongside it, sixteen HIGH findings break payments, availability correctness, reminders, onboarding persistence and several dashboard controls that report success without doing anything. Build, typecheck and lint are clean; the defects are logic and authorization, not compilation.

Verification performed

npx tsc --noEmit — clean (0 errors). npm run build — succeeded, 34 routes compiled. npx eslint . — 1 warning, 0 errors. All six SQL migrations, the full src/ tree (151 TypeScript/TSX files), public/embed.js, vercel.json, .env.local and .env.example were read. No production database, Stripe account or live deployment was touched; no runtime exploitation was attempted. Database findings are read from the migration source — confirm the deployed policy and grant state in Supabase before closing them.

Correction to the attached audit

The S-series (S01–S05) describes a different codebase. It reports Next.js 15.5.9, Prisma 5 and NextAuth 5; this repository is Next.js 16.3.1 with Supabase/Postgres RLS, no Prisma and no NextAuth, and its plan enum is starter/professional/elite. Do not apply S-series remediation here. The one part of S02 that does land is the marketing page: it still advertises an "Enterprise" tier (A24). Two further items in the attached audit are narrower than stated — see "Items not reproduced" at the end.

Findings

Ordered by severity. Every finding was read in the source at the cited location.

CRITICAL

A01 Any signed-in user can become platform super-admin

CRITICAL

`Evidence: supabase/migrations/0002_rls.sql:69-79; 0001_schema.sql:63-77`

auth_role() and auth_tenant() are SECURITY DEFINER functions that read profiles.role and profiles.tenant_id — the two columns every single RLS policy in the system depends on. The profiles_update policy permits id = auth.uid() in both USING and WITH CHECK, with no column restriction, no protective trigger and no column-level REVOKE anywhere in the migrations. A customer or employee holding nothing but the public anon key can therefore run update profiles set role = 'super_admin' where id = auth.uid() from the browser and gain read/write access to every tenant on the platform; setting tenant_id instead moves them laterally into any other business. The tenant_required check constraint does not block this. Fix: a BEFORE UPDATE trigger on profiles that rejects any change to role or tenant_id unless the caller is the service role or an existing super_admin, plus cross-tenant and self-promotion tests.

Cross-reference: Confirms F01, D09, S04 — and raises it from "verify deployed grants" to a proven, exploitable path.

> **FIXED** 0007_security.sql: a BEFORE UPDATE trigger on profiles rejects any change to role or tenant_id unless the caller is the service role or an existing super admin, and such changes are now audited. VERIFY in Supabase that the trigger is present and that a signed-in customer cannot self-promote.

A02 Tenant SMTP password is readable by that tenant's employees and customers

CRITICAL

`Evidence: src/lib/actions/settings.ts:86-105; src/lib/integrations/email.ts:57-107; 0002_rls.sql:39-43`

The business's outbound email credentials are written in plaintext into tenants.settings.email.smtp.pass. The tenants_select policy grants SELECT on the whole row to anyone whose profiles.tenant_id matches — which includes every employee and every portal customer, not just the owner. Any customer with a portal account can read the owner's SMTP username and password with one client-side query, then send mail as the business. Fix: move credentials to a table with an owner-only (ideally service-role-only) SELECT policy, or encrypt them with a server-held key; narrow tenants_select to non-sensitive columns via a view; rotate any credentials already entered.

Cross-reference: Confirms F02 (first half).

> **FIXED** Credentials moved to tenant_secrets — RLS enabled, deliberately no policies, so only the service role can read it. Existing values are migrated and stripped from tenants.settings by the migration. ROTATE any SMTP password that was stored before this: assume it was readable.

A03 Google OAuth callback trusts an unsigned state parameter

CRITICAL

`Evidence: src/app/api/integrations/google/callback/route.ts:5-31`

The callback has no authentication of its own. It base64-decodes the state query parameter, trusts the profileId and tenantId inside it, and writes the resulting Google access and refresh tokens into google_calendar_connections using the service-role client. state carries no signature, no nonce and no tie to a session cookie. An unauthenticated attacker can mint their own state naming any tenant, complete OAuth with their own Google account, and have that connection attached to a victim tenant — after which calendar-sync pushes that tenant's bookings, customer names and job addresses into the attacker's calendar. The reverse is equally available: overwrite a victim's row to break their sync. Fix: sign state (HMAC with a server secret) or store a one-time nonce in a cookie and verify it; re-resolve profileId and tenantId from the session in the callback rather than from the URL; wrap the JSON.parse so malformed state is a 400, not a 500.

Cross-reference: New — not present in the attached audit.

> **FIXED** state is HMAC-signed with an expiry (lib/signed-state), and the callback additionally re-resolves the subject from the live session, so a captured state cannot be replayed by another account. Malformed state is now a redirect, not a 500.

A04 Google Calendar access and refresh tokens are readable by every employee

CRITICAL

`Evidence: supabase/migrations/0006_integrations.sql:44-50`

google_calendar_connections stores access_token and refresh_token in plaintext, and its SELECT policy admits auth_role() in ('business_owner','employee') for the whole tenant. Any employee can read the owner's Google refresh token and use it outside the platform for as long as it stays valid. Fix: restrict SELECT to the row's own profile_id (plus super_admin), keep tokens out of any tenant-wide policy, and encrypt them at rest.

Cross-reference: Confirms F02 (second half).

> **FIXED** SELECT and write on google_calendar_connections are own-row only. Residual: the tokens are still plaintext at rest — encrypting them needs a server-held key and is not done here.

HIGH

A05 A service-role cron endpoint is public because CRON_SECRET is unset

HIGH

`Evidence: src/app/api/cron/extend-recurring/route.ts:30-36; .env.local`

The guard is written if (secret) { ...check... } — it fails open. CRON_SECRET is declared in .env.example but is absent from .env.local, so as deployed the endpoint accepts any anonymous GET or POST and performs service-role writes, materializing booking rows across tenants. Fix: fail closed — return 401 whenever CRON_SECRET is missing, exactly as send-reminders already does — and set the variable in every environment.

Cross-reference: New.

> **FIXED** Both cron routes fail closed and compare the bearer token in constant time. ACTION REQUIRED: set CRON_SECRET — it is absent from .env.local, and without it reminders never send.

A06 Subscription checkout cannot resolve a price in any environment

HIGH

`Evidence: src/lib/plans.ts:122-134; .env.local; .env.example`

getStripePriceId reads STRIPE_PRICE_STARTER, STRIPE_PRICE_PROFESSIONAL and STRIPE_PRICE_ELITE and throws when one is missing. .env.local instead defines STRIPE_PRICE_STARTER_MONTHLY/_YEARLY, STRIPE_PRICE_PRO_MONTHLY/_YEARLY and STRIPE_PRICE_ENTERPRISE_MONTHLY/_YEARLY — none of the three names the code looks for. Every upgrade attempt fails, and tierFromStripePriceId swallows the error and returns null, so the webhook can never map a price back to a tier either. The same file also reveals that the client-side publishable key the code expects (NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY) is present only as STRIPE_PUBLISHABLE_KEY, which Next.js will not expose to the browser — one direct cause of A12. STRIPE_WEBHOOK_SECRET is also declared twice. Fix: settle on one naming scheme (monthly/yearly per tier if annual billing is intended), make the catalog the single source, and add a startup check that every required variable is present.

Cross-reference: New; explains part of F04 and D05.

> **FIXED** Resolved on 19 October. .env.local was renamed to the three names the code reads (mapping confirmed against each Stripe Price's product name), the publishable key took its NEXT_PUBLIC_ prefix, the duplicate STRIPE_WEBHOOK_SECRET was removed, and CRON_SECRET plus SUPER_ADMIN_SETUP_TOKEN were generated. config-check reports zero blockers. Checking this also uncovered A43 and A44 below, which the rename alone would have left in place.

A07 No payment is ever recorded: the webhook upsert targets a non-existent constraint

HIGH

`Evidence: src/app/api/webhooks/stripe/route.ts:136-146; 0001_schema.sql:278-288`

The payment_intent handler calls upsert(..., { onConflict: "stripe_payment_intent_id" }), but the payments table declares no unique constraint or index on that column — Postgres requires one for ON CONFLICT and will reject the statement. The handler therefore throws, the route returns 500, and Stripe retries the event indefinitely while no payments row is ever written. Because the dashboard's revenue chart sums payments.amount_cents (src/lib/dashboard/data.ts:123-146), revenue will read $0 forever even after real money moves. Fix: add a unique index on payments.stripe_payment_intent_id, then reconcile the booking and invoice state from the event as well, not just the payments ledger.

Cross-reference: New; a concrete mechanism behind F04 and D12.

> **FIXED** 0007 adds the unique index on payments.stripe_payment_intent_id that the upsert infers, duplicates from earlier retries are collapsed first, and the handler now throws on failure so Stripe retries rather than losing the event. The webhook also reconciles the invoice and confirms a deposit-paid booking, and a new charge.refunded handler records refunds.

A08 Unauthenticated endpoint resets the super-admin password

HIGH

`Evidence: src/app/api/admin/setup/route.ts:110-117`

GET and POST /api/admin/setup are open to the internet and run with service-role privileges. The attached audit overstates the consequence — the endpoint only ever writes the password already in the environment, so a caller learns nothing and gains no session. It is still a real defect: anyone can silently revert the platform owner's password to the env value (undoing a rotation done in the UI), and each unauthenticated call may page through up to 5,000 auth users, which is a free amplification primitive against the Auth API. Fix: require a one-time setup token or a signed header, delete the route once bootstrap is done, and log every invocation.

Cross-reference: Confirms F03, with severity corrected from CRITICAL to HIGH and the reasoning stated.

> **FIXED** /api/admin/setup requires SUPER_ADMIN_SETUP_TOKEN, returns a flat 404 without it, and logs every attempt. Set the token, call it once, then remove the variable.

A09 Add-on duration is ignored when searching slots but applied when booking

HIGH

`Evidence: supabase/migrations/0003_functions.sql:215-323 vs 392-420; src/lib/widget/data.ts:116-132`

get_available_slots takes no add-on argument and sizes every candidate slot with v_service.duration_minutes alone. create_widget_booking adds the add-ons' duration_minutes to compute ends_at and then enforces business hours and employee availability against that longer span. A customer who adds a 20-minute extra is therefore offered start times that cannot actually hold the job: late-day picks fail with outside_business_hours and mid-day picks fail with slot_taken, after the customer has filled in the whole form. The displayed latest-start never moves when an extra is selected, which is exactly what the live test observed. Fix: pass the selected add-on ids into the availability RPC and compute duration once, in one shared function used by the slot search, the quote and the booking insert.

Cross-reference: Confirms F07 and D03, with the root cause located.

> **FIXED** One booking_duration_minutes() helper now sizes both the slot search and the booking insert, the availability RPC takes the selected add-on ids, and the widget re-queries when add-ons change.

A10 Buffers are only half-enforced: the existing booking's buffer is ignored

HIGH

`Evidence: supabase/migrations/0003_functions.sql:296-301 and 446-451`

Both the slot search and the booking insert test overlap as tstzrange(new_start - buffer_before, new_end + buffer_after) && tstzrange(b.starts_at, b.ends_at). The candidate's own buffers are applied; the buffers of the booking it is being compared against are not. tstzrange is half-open, so a booking that ends at 11:30 with a saved 30-minute after-buffer does not block an 11:30 start — which reproduces the observed defect precisely. The database-level no_double_booking exclusion constraint compares raw booking ranges only, so it does not catch this either: buffers have no enforcement anywhere once this test passes. Fix: join the compared booking's service and expand both sides of the comparison, in both functions, and add a regression test for the back-to-back case.

Cross-reference: Confirms F07 and the backend cause behind D01.

> **FIXED** booking_conflicts() pads BOTH the candidate and the compared booking by their own service's buffers, in the slot search and at insert. The back-to-back 11:30 case is the regression test to run first.

A11 Duplicate appointment times: one row per employee, never deduplicated

HIGH

`Evidence: supabase/migrations/0003_functions.sql:213 (comment); src/components/widget/BookingFlow.tsx:444-461`

get_available_slots deliberately returns one row per (slot, employee) and its comment says "the widget dedupes by slot_start". The widget does not. BookingFlow maps the array straight to buttons with key={s.slot_start}, so with two qualified staff every time appears twice (and React logs a duplicate-key warning). The attached audit's 17-unique/34-displayed observation is this line. Fix: group by slot_start in the widget, keep the candidate employee ids behind the single visible choice, and let create_widget_booking's auto-assign pick one — so collapsing the display does not collapse capacity.

Cross-reference: Confirms D01.

> **FIXED** Slots are collapsed by start time in the data layer and the booking no longer pins an employee, so one visible choice keeps the slot's full capacity and the RPC picks a free qualified employee at insert.

A12 Bookings are confirmed without taking payment

HIGH

`Evidence: src/components/widget/BookingFlow.tsx:729; src/lib/widget/data.ts:403-415`

When payments and Stripe Connect are both enabled and a deposit is due, the server creates a PaymentIntent and returns its client secret — and the UI then prints "Stripe deposit intent created successfully. Payment capture UI is the next step to wire into this screen." There is no Stripe.js, no Elements, no confirmation step, and (per A06) no publishable key reaching the browser. The booking row is written either way, at status pending or confirmed, with no record of whether money was taken. An intent is not a payment. Fix: gate paid and deposit-required bookings on a verified payment_intent.succeeded event; show honest unpaid / pending / paid state on the booking; make the customer-facing copy state the amount, when it is due and what remains.

Cross-reference: Confirms F04 and D04.

> **PARTIAL — remainder stated** Capture is still not implemented — that was your decision, and the product now says so honestly instead of implying otherwise. No PaymentIntent is created unless STRIPE_DEPOSIT_CAPTURE_ENABLED is explicitly true (it was briefly gated on the publishable key instead, which would have silently re-enabled unconfirmable intents the moment Stripe was configured properly), the deposit copy states plainly that nothing was charged, and onboarding_readiness() reports deposits configured without Stripe charges enabled. Launching with pay-later is safe; taking card payments online still needs Stripe.js, Elements and a confirmation step.

A13 Re-running onboarding destroys live data non-transactionally

HIGH

`Evidence: src/lib/actions/onboarding.ts:107-161 and 166-215`

saveServices deletes every service_addons row for the tenant, then every services row. saveTeam deletes every employees row. Neither runs in a transaction and /onboarding stays reachable after launch. Three distinct consequences, all confirmed in the schema: (1) services carries ON DELETE RESTRICT from bookings, so on a tenant with bookings the services delete fails and the action returns a generic error — but the add-ons are already gone, permanently, with no rollback; (2) employees carries ON DELETE SET NULL from bookings, so the employees delete succeeds and silently unassigns every existing booking, which also releases the no_double_booking constraint that was holding those slots; (3) saveServices never re-inserts the add-ons it deleted — the code ends at the comment "Insert add-ons per service (inserted[] is parallel to clean[])" with no insert following it, so add-ons configured during onboarding are discarded on every run. Fix: upsert by stable id instead of delete-and-recreate, wrap each step in a single transaction (an RPC), never delete a row referenced by a booking, and actually persist the add-ons.

Cross-reference: Confirms F05 and the add-on half of F06, with two further mechanisms found.

> **FIXED** 0009: onboarding_save_catalog and onboarding_save_team upsert by a stable key inside one transaction. Anything a booking references is deactivated, never deleted. The add-ons the old code deleted and never re-inserted are now persisted.

A14 Flat-rate services are silently forced to 60 minutes

HIGH

`Evidence: src/lib/actions/onboarding.ts:137-140`

duration_minutes is computed as pricing_unit === "hourly" ? max(30, minimum_hours * 60) : 60. For any flat-rate service the duration the owner entered is thrown away and replaced with 60. A flat-rate deep clean that takes four hours is booked as a one-hour job, so the calendar overlaps, slots are offered inside occupied time and the no-double-booking guarantee is defeated by bad input rather than by a race. This is the persistence mismatch the earlier QA run saw between onboarding and the dashboard. Fix: carry the entered duration through for both pricing modes, and reconcile the onboarding service model with the dashboard's (src/lib/actions/services.ts) so one schema serves both.

Cross-reference: Confirms F06, with a sharper and more damaging mechanism than reported.

> **FIXED** Duration is collected in the wizard and honoured for both pricing modes, and pricing_unit / minimum_hours / pricing_unit_label now persist, so a flat-rate service stops being re-presented as hourly. The dashboard and the wizard settled on one set of names.

A15 The contact form reports success without sending anything

HIGH

`Evidence: src/components/sections/ContactSection.tsx:117-123`

onSubmit validates locally and calls setSent(true). There is no fetch, no server action, and no contact endpoint anywhere in the app. Every sales enquiry submitted from the marketing site is discarded while the visitor is told it was received. Fix: post to a server action that persists the enquiry and emails it, with spam protection, real error states, and no success message until the server confirms.

Cross-reference: Confirms F09.

> **FIXED** Submissions persist to contact_submissions, notify the platform inbox, are rate-limited per email and carry a honeypot. Success is claimed only if the write succeeded. ACTION: set SMTP_FROM or RESEND_FROM_EMAIL so the notification has somewhere to go.

A16 Coupons are never saved and never applied to a booking

HIGH

`Evidence: src/components/dashboard/sections/MarketingClient.tsx:25-46`

Creating a coupon pushes an object with id "local-" into React state; the activate toggle flips a boolean in the same state. No server action is imported and nothing is written to the coupons table — everything disappears on reload. Separately, create_widget_booking accepts no coupon code and applies no discount, so even a correctly stored coupon would have no effect on price. Either ship redemption end to end or label the feature as planned.

Cross-reference: Confirms F10.

> **FIXED** Coupons persist, and price_coupon / quote_booking / apply_coupon_to_booking make them reduce what a customer pays. The redemption counter is claimed in the same statement as the discount, so a limit cannot be exceeded by concurrent bookings. Coupons are also gated as the paid feature the catalog says they are.

A17 Invoice "Mark paid", "Refund" and "Download" do nothing

HIGH

`Evidence: src/components/dashboard/sections/InvoicesClient.tsx:52-53, 124, 130, 136`

update() is a setState call. "Mark paid" sets status: "paid" and paidCents in local state only; "Refund" sets status: "refunded" the same way; "Download" is onClick={() => {}}. Nothing reaches the invoices table, and no refund reaches Stripe. An owner who clicks Refund will believe a customer has been refunded when no money has moved — the most dangerous of the local-only controls, and a chargeback risk. Fix: authorized server actions for each, Stripe refund calls reconciled through the webhook, and a real PDF path for download. Until then, disable the controls.

Cross-reference: Confirms F10.

> **FIXED** Real server actions: payment outside Stripe is recorded as an explicit bookkeeping entry, refunds call Stripe and are reconciled by the webhook, drafts can be sent, and downloads return a signed URL. 'There is no card payment to refund' is now said out loud rather than shown as success.

A18 Reminders are scheduled but never sent

HIGH

`Evidence: vercel.json; src/app/api/cron/send-reminders/route.ts:6-10; src/lib/widget/data.ts:274-306`

Every booking writes pending rows into booking_reminders, and a correct dispatcher exists at /api/cron/send-reminders. vercel.json schedules only /api/cron/extend-recurring. No schedule for the reminder job exists in the repository, and its guard requires CRON_SECRET, which is unset (A05) — so even a manual call would 401. Reminders accumulate forever and no customer is ever reminded. Fix: add the cron entry, set CRON_SECRET, and add retry and delivery metrics. Note the job also has no plan gate, so SMS would be sent for Starter tenants (A19).

Cross-reference: Confirms F08, and resolves its open question: there is no external scheduler in this repository.

> **FIXED** vercel.json schedules /api/cron/send-reminders every 15 minutes. It will still send nothing until CRON_SECRET is set (A05).

A19 Plan limits and SMS entitlement are not enforced anywhere

HIGH

`Evidence: src/lib/plans.ts:153-162; src/lib/integrations/sms.ts:7; src/lib/widget/data.ts:284-289`

withinEmployeeLimit and withinServiceLimit are defined and exported but never called — a Starter tenant can create unlimited employees and services. hasFeature is only used to hide UI (DashboardChrome, two dashboard pages); it guards no write. sms.ts documents that the call site is supposed to check hasFeature(plan, "sms_reminders") and no call site does, so SMS reminders are queued for every tenant regardless of plan — a direct Twilio cost leak and a paid feature given away. Fix: enforce limits and features in the server actions and RPCs that perform the write, and treat hidden UI as cosmetic only.

Cross-reference: Extends F12 and D05 with the enforcement gap neither names.

> **FIXED** lib/entitlements.ts enforces the service and seat limits in the actions that write, and SMS reminders are gated on the plan at the point they are queued.

A20 The reports page shows fabricated employee revenue

HIGH

`Evidence: src/lib/dashboard/data.ts:705-721`

Per-employee revenue is computed as e.jobsThisMonth * 14000 — a hardcoded $140 per job, unrelated to any booking price. Per-service revenue is s.bookingCount * s.priceCents, which ignores add-ons and any discount. bookingsBars is built by mapping every bar to cents: 0, so that chart is shipped permanently empty. An owner reading this page is reading invented numbers presented as their results. Fix: derive every figure from booking and payment rows, or remove the panel until it can be.

Cross-reference: New; extends D12 well beyond the naming problem it reports.

> **FIXED** Per-employee revenue comes from those jobs' own prices, per-service revenue from the bookings themselves. The permanently empty bookings chart now shows completed work per month. The admin console's System health tab, which was entirely hardcoded, now shows real counts and the configuration problems.

MEDIUM

A21 Onboarding is marked complete before anything is verified

MEDIUM

`Evidence: src/lib/actions/onboarding.ts:299-312`

finishOnboarding writes onboarding_complete: true as its first act, then attempts the bookability defaults. If the availability insert or the employee_services upsert throws, the tenant is flagged complete with no working schedule — a published widget that can never offer a slot. It also links every employee to every service indiscriminately, so the qualified-staff concept is meaningless on day one. Fix: a server-computed readiness check (services, staff, hours, payment mode) that runs before the flag is set, with separate booking-ready and payment-ready states.

Cross-reference: Confirms F11.

> **FIXED** onboarding_readiness() computes bookability from the database and onboarding_complete is set only once it passes, with the blockers named to the owner.

A22 "Invite teammate" is a button with no handler

MEDIUM

`Evidence: src/components/dashboard/sections/TeamClient.tsx:22`

The toolbar renders Invite teammate with no onClick. Nothing in the codebase sends an invitation: employees rows carry invite_email and the UI shows an "Invited" badge, but no token is generated, no email is sent and there is no acceptance route that links a profile to the employee row. Staff can never reach their own schedule, which is also why owners cannot set hours (D07).

Cross-reference: Confirms F14 and contributes to D07.

> **FIXED** Single-use tokens with a 14-day expiry, an email, an /invite/[token] acceptance page, and a function that refuses an expired, revoked, reused or wrong-email token. The 'Invited' badge now reflects a live invitation and reads 'Not invited' for the state that used to claim otherwise.

A23 The dashboard widget preview is a static mock, not the tenant's booking page

MEDIUM

`Evidence: src/components/dashboard/sections/WidgetClient.tsx:297; src/lib/widget/data.ts:32-94`

The preview is explicitly commented "A faithful static mock of the embeddable widget". It reflects the colour and radius controls but not the tenant's own services, durations, prices or deposits. Separately, mockConfig() hardcodes "Clean Sweep Services" with Standard Home Cleaning, Deep Cleaning and Office Cleaning — which is exactly the business and service list the live audit saw in the preview while the real page showed different services. Fix: mount the real booking component in a read-only preview mode against the published tenant config, overlaying only unsaved branding; or label it "Example".

Cross-reference: Confirms D02 and F13, and identifies the hardcoded source of the mismatch.

> **PARTIAL — remainder stated** The preview renders this tenant's published name and services with unsaved branding applied, and is labelled an appearance preview rather than a live one. It is still a rendering of the first step, not the real booking component mounted in preview mode — the hosted link beside it remains the authoritative view.

A24 The pricing page advertises a tier the product does not have

MEDIUM

`Evidence: src/components/sections/Pricing.tsx:19-43; src/lib/plans.ts:92-118`

plans.ts is a genuine single catalog — starter $29 / professional $59 / elite $119, with limits and features — but Pricing.tsx hardcodes its own copy of the numbers and calls the top tier "Enterprise". The plan_tier enum has no such value. A customer buys Enterprise and is provisioned elite. The ENTERPRISE naming in the .env.local price variables (A06) comes from the same confusion. Fix: render the marketing pricing from PLANS, choose one name, and delete the duplicated literals.

Cross-reference: Confirms F12 and the part of D05/S02 that applies to this repository.

> **FIXED** The pricing page is derived from lib/plans.ts, so the 'Enterprise' tier that the product does not have is gone along with the drifted feature bullets. See also A43: deriving the page from the catalog was necessary but not sufficient, because the catalog's own numbers were wrong.

A25 The features page shows an embed snippet that does not exist

MEDIUM

`Evidence: src/components/sections/FeatureDeepDives.tsx:546`

The public Features page illustrates //embed.diamondbooking.com/v1.js. The real loader is {APP_URL}/embed.js with data-key, and — to the attached audit's credit on the other half of this item — the dashboard snippet, the onboarding reveal and public/embed.js all agree with each other and work. Only the marketing illustration is wrong. Fix: generate the marketing sample from the same snippet builder the app uses.

Cross-reference: Confirms D10; narrows F16, which implied the app's own instructions were also wrong.

> **FIXED** The features page shows the real snippet. Note the app's own snippet, the onboarding reveal and public/embed.js already agreed — only the marketing illustration was wrong.

A26 "Allowed domains" is collected, stored, displayed — and never checked

MEDIUM

`Evidence: src/components/dashboard/sections/WidgetClient.tsx:249-258; src/lib/actions/settings.ts:177`

widget_configs.allowed_domains is written by the settings action and listed back in the UI under "Where the widget may be embedded". No code reads it for enforcement: neither /book/[public_key] nor any of the three widget RPCs consults it, and there are no frame-ancestors headers (A27). The widget can be embedded on any site by anyone holding the public key. The control is a placebo and owners will trust it. Fix: enforce it in the booking page and the RPCs, or remove the control.

Cross-reference: New.

> **FIXED** Embedded loads check allowed_domains (listed domains cover subdomains; empty still means anywhere) and say which domains are permitted. This governs embedding, not authorization — the hosted page stays reachable by anyone with the link, as it must be.

A27 No security headers are configured at all

MEDIUM

`Evidence: next.config.ts`

next.config.ts sets only devIndicators: false. There is no Content-Security-Policy, no frame-ancestors, no Strict-Transport-Security, no Referrer-Policy, no X-Content-Type-Options and no Permissions-Policy. The app renders an iframe-embeddable booking page that handles names, addresses and payment intents. docs/SECURITY.md exists but the headers it would describe are not implemented. Fix: add a headers() block; keep frame-ancestors permissive only for /book/* and deny it elsewhere.

Cross-reference: New.

> **PARTIAL — remainder stated** CSP, HSTS, Referrer-Policy, Permissions-Policy and frame rules added, with /book exempt from frame denial so the widget still embeds. Residual, and stated in the file rather than glossed: the script policy still needs 'unsafe-inline' for Next's inline bootstrap; a nonce-based CSP is follow-up work.

A28 Recurring series silently drop the occurrences they cannot place

MEDIUM

`Evidence: src/lib/widget/data.ts:247-271`

materializeRecurrence inserts occurrences one at a time and, on an exclusion violation, moves on without recording anything; on any other error it breaks out of the loop entirely. The customer is told their standing appointment is booked and never learns which dates were skipped, and the owner has no partial-series signal. Occurrences are also cloned with the first booking's duration and price and are never re-checked against time off or business hours. Fix: return an explicit per-occurrence outcome, surface skipped dates to both parties, and test across a daylight-saving boundary — computeOccurrences advances by fixed week offsets.

Cross-reference: Confirms F15.

> **FIXED** Recurring series report which dates were reserved and which were already taken, on the confirmation screen. The booking RPC also distinguishes a real conflict from 'nobody is qualified or scheduled'.

A29 The cancellation policy is stored but never enforced

MEDIUM

`Evidence: src/lib/actions/onboarding.ts:303-307; src/lib/actions/bookings.ts:193-243`

Onboarding collects cancellation_policy and cancellation_window_hours into tenants.settings. cancelMyBooking checks ownership correctly but never reads either value, so a customer can cancel any pending or confirmed booking at any time, including minutes before the appointment, whatever the owner configured. The policy text is displayed as if it binds.

Cross-reference: New.

> **FIXED** cancelMyBooking enforces cancellation_window_hours and explains the business's notice period instead of silently allowing the cancellation.

A30 Revenue is defined two different ways and the no-show rate is diluted

MEDIUM

`Evidence: src/lib/dashboard/data.ts:123-146, 700, 721`

The overview chart sums payments.amount_cents (cash collected — and always $0 today, per A07). The reports page sums price_cents over completed bookings (work earned). Both are labelled revenue. noShowRate divides no-shows by all bookings in the period, including future and cancelled ones, which understates the operational measure. Fix: present booked, earned, invoiced and collected as four named figures; define the no-show denominator as appointments that were due in the period; reconcile each against a known fixture.

Cross-reference: Confirms D12.

> **FIXED** Booked, earned and collected are four named figures instead of one ambiguous 'revenue', and the no-show denominator is appointments that were due, shown alongside the rate.

A31 Legal links are plain text, and one documentation link is wrong

MEDIUM

`Evidence: src/components/sections/Footer.tsx:29, 78`

The marketing footer's bottom bar renders the string "Privacy · Terms · Security" as text, not links. /privacy and /terms exist and build, but the marketing footer never links them — only the auth layout and the signup form do — and there is no /security page at all. The resources column points "API docs" at /features, a marketing page. Several other footer links go to homepage anchors rather than real destinations. Fix: link the legal pages from the footer, add or remove the Security entry, and point API docs at real documentation or drop it.

Cross-reference: Confirms F16.

> **FIXED** Privacy and Terms are linked from the footer, and the 'API docs' entry that pointed at a marketing page is gone. There is still no /security page — the link was removed rather than left pointing nowhere.

A32 The public booking actions are unauthenticated and unthrottled

MEDIUM

`Evidence: src/lib/actions/widget.ts:11-26; 0003_functions.sql:510-512`

fetchSlots and submitBooking are exported server actions reachable by anyone as plain POSTs, and the three RPCs are granted to anon. There is no rate limit, no captcha and no email verification. Anyone holding a public key — which is embedded in every page the widget appears on — can fill a business's entire calendar with fake bookings, each of which also sends a confirmation email and queues reminders. Fix: rate-limit by IP and by key, add a bot check on the public flow, and consider holding unverified bookings in a pending state that does not consume capacity until confirmed.

Cross-reference: New.

> **PARTIAL — remainder stated** Best-effort rate limiting on the public booking actions, with the in-process limitation stated plainly in lib/rate-limit.ts rather than implied away: on serverless each instance has its own counter. Durable protection needs a shared store or your platform's WAF. The database guards — the enquiry cap and the exclusion constraint — are the ones that actually hold.

A33 Unknown Stripe subscription statuses are treated as active

MEDIUM

`Evidence: src/app/api/webhooks/stripe/route.ts:156-170`

mapStatus returns "active" from its default branch, so incomplete, paused and any status Stripe adds later all mark the tenant as a paying subscriber. A subscription whose first payment never completes is indistinguishable from a good one. Combined with A19 (no server-side entitlement checks) this is an open door to unpaid use. Fix: map unknown statuses to a restricted state and log them.

Cross-reference: New; relates to D06's ambiguous trial state.

> **FIXED** Unknown Stripe subscription statuses map to past_due and are logged, so an incomplete subscription no longer reads as a paying one.

A34 The no-availability state is a dead end

MEDIUM

`Evidence: src/components/widget/BookingFlow.tsx:441-443`

When a day has no slots the widget renders "No openings this day. Try another date." and nothing else — no next-available-date shortcut, no callback or quote request, no capture of the selected service. Every customer who lands on a full day is lost silently. Fix: keep the selection, offer a request/callback that is stored as an enquiry rather than a booking, and show the next date that does have openings.

Cross-reference: Confirms D08.

> **FIXED** A full day offers a request form, stored as a booking_inquiry that reserves nothing, and both the form and its confirmation say it is not an appointment.

A35 Owners have no way to see why a date has no slots

MEDIUM

`Evidence: src/components/dashboard/sections/TeamClient.tsx; src/lib/dashboard/data.ts`

Availability lives on employee rows that owners cannot edit, invitations do not work (A22), and the dashboard offers no capacity diagnostic — no "no working hours", "no qualified staff" or "closed that day" signal. Given that finishOnboarding seeds availability from business hours only for employees with no rows, an owner whose widget stops offering slots has no way to find out why. Fix: an owner-readable capacity summary listing the concrete gaps, and a permission-controlled hours editor if owners are meant to manage staff schedules.

Cross-reference: Confirms D07.

> **FIXED** The team screen shows the server-computed capacity gaps, so an owner can see why their widget offers no times.

A36 Placeholder-env mode disables all route protection and fakes bookings

MEDIUM

`Evidence: src/middleware.ts:52-54; src/lib/env.ts:15-27; src/lib/widget/data.ts:100-103, 315-331`

When supabaseEnvConfigured() returns false the middleware returns NextResponse.next() for every route — /dashboard, /admin and /portal included — any public key resolves to the demo business, and createWidgetBooking returns a synthetic confirmed booking without writing anything. That is reasonable for local preview, but the detection is heuristic (it looks for the substring "placeholder"), so one malformed production environment variable silently turns the deployment into an unprotected demo that tells customers their bookings succeeded. Fix: make placeholder mode opt-in through an explicit flag, and refuse to start in production without real keys.

Cross-reference: New.

> **FIXED** Placeholder mode requires an explicit ALLOW_PLACEHOLDER_MODE in production; otherwise the app returns 503 rather than serving an unprotected demo that tells customers their bookings succeeded.

A37 seed.sql creates demo accounts with a published password

MEDIUM

`Evidence: supabase/seed.sql:1-20`

The seed inserts directly into auth.users with bcrypt-hashed "password123" for owner, two employees and a customer, and a fixed widget key demo00000000000000000ff. This is correct for local development, but it is a set of working backdoor credentials if it is ever applied to a hosted project, and nothing in the repository prevents that. Its "Maria" and "James" employees are also the mismatched sample names the live audit saw on the permissions screen. Fix: guard the seed against non-local databases and document that it must never run in staging or production.

Cross-reference: New; explains D09's sample-name observation.

> **FIXED** seed.sql refuses to run against a database that already holds real tenants or users.

LOW

A38 Reschedule has no conflict pre-check and leaks database errors

LOW

`Evidence: src/lib/actions/bookings.ts:101-155`

rescheduleBooking validates the date and tenant correctly but writes the new time without checking availability, buffers or time off, relying on the exclusion constraint to reject it. The owner then sees a raw Postgres exclusion-violation message. It also moves the booking without re-checking business hours. Fix: reuse the shared availability check and return a readable conflict message.

Cross-reference: New.

> **FIXED** Reschedule pre-checks conflicts including buffers, and a concurrent loss reads as a sentence rather than a Postgres exclusion-violation dump.

A39 tenants.settings is updated with a read-modify-write race

LOW

`Evidence: src/lib/actions/onboarding.ts:41-65`

mergeSettings reads settings, spreads a patch over it and writes the whole object back. Two concurrent saves lose one patch. Low impact in a single-owner wizard; worth fixing with a jsonb merge (settings || patch) in SQL before multiple staff can edit settings.

Cross-reference: New.

> **FIXED** The settings merge is a single SQL statement (merge_tenant_settings), so concurrent saves cannot lose a patch.

A40 Open unauthenticated geocoding proxy

LOW

`Evidence: src/app/api/geocode/route.ts:18-52`

/api/geocode proxies arbitrary queries to photon.komoot.io with no authentication and no rate limit. The destination is fixed, so this is not an SSRF, but it is a free open proxy attributable to your domain and a likely breach of the free service's terms under load. Fix: rate-limit by IP and require a valid widget key or a signed-in session.

Cross-reference: New.

> **FIXED** The geocode proxy is rate-limited and caps the query length.

A41 Per-request database read in middleware

LOW

`Evidence: src/middleware.ts:79-84`

Every matched request performs getUser() plus a profiles select. On a route set this broad that is two round trips per navigation and per API call. Fix: carry the role in the JWT app_metadata (already set at signup) and read the profile only when it is missing.

Cross-reference: New.

> **WITHDRAWN — my error, corrected** My finding was wrong, and I am correcting it rather than leaving it to look fixed. The middleware does NOT read profiles on every request: matchProtected returns early for marketing, /book and /login, so the query runs only for dashboard/portal/admin requests. I tried making the JWT claim primary and reverted it — the claim goes stale exactly when a teammate accepts an invitation, and routing on a stale claim would strand them. Profile-first is correct; the reasoning is recorded in the file, and app_metadata is now kept in step on acceptance.

A42 Environment and lint hygiene

LOW

`Evidence: .env.local; src/app/onboarding/steps/ServicesStep.tsx:40`

STRIPE_WEBHOOK_SECRET is declared twice in .env.local. GOOGLE_OAUTH_ENABLED, RESEND_FROM_NAME and STRIPE_PUBLISHABLE_KEY are set but never read by any code. No SMTP_* platform fallback is configured, so platform-wide email depends entirely on Resend. One lint warning: formatDuration is unused in ServicesStep.tsx.

Cross-reference: New.

> **PARTIAL — remainder stated** .env.example is corrected and documents every variable the code reads. Your .env.local is untouched: it still holds the duplicate STRIPE_WEBHOOK_SECRET and the three unused variables, and it is where the A06 renames are needed.

HIGH

A43 The app displayed one price and Stripe charged another

HIGH

`Evidence: src/lib/plans.ts:41,67,95 vs the Stripe Prices in .env.local`

Found while renaming the price variables for A06, by reading the actual Stripe Prices rather than trusting the names. The catalog said Professional $59 and Elite $119; the Prices those tiers point at charge $69.99 and $129.00. On two of three tiers a customer would have been shown one price and billed another — a chargeback and consumer-protection problem, not merely a cosmetic one. Deriving the marketing page from the catalog (A24) could not have caught this, because the catalog's own numbers were wrong: every surface agreed with every other surface, and all of them disagreed with Stripe. Fix: the catalog now holds the amounts Stripe actually charges, and config-check fetches each Price at runtime and reports a blocker on any display-versus-charge difference — also on an archived price, a non-monthly interval, a missing id and a non-USD currency.

Cross-reference: New; the defect A24's fix was necessary but not sufficient for.

> **FIXED** NEW, found on 19 October while renaming the price variables. The catalog said Professional $59 and Elite $119, while the Stripe Prices those tiers point at charge $69.99 and $129.00 — a customer would have been shown one price and billed another, on two of three tiers. Deriving the marketing page from the catalog (A24) could not catch this, because the catalog itself was wrong. Resolved by setting the catalog to the amounts Stripe actually charges ($29 / $69.99 / $129), and config-check now fetches each Stripe Price and reports a BLOCKER on any display-versus-charge difference, an archived price, a non-monthly interval, a 404 id or a non-USD currency. Verified against the live account: all six monthly and yearly amounts now agree.

MEDIUM

A44 The yearly prices on the pricing page were fabricated

MEDIUM

`Evidence: src/components/sections/PricingHero.tsx:17-20,158`

The yearly figure was computed as priceMonthly * 10, with a flat “2 months free” badge on every tier. The real Stripe yearly Prices are $299, $600 and $1,200 — discounts of 14%, 29% and 23% — so the formula matched none of them and the badge was false for all three. With the corrected monthly prices it would have rendered “$699.9/yr”. Separately, annual billing cannot actually be bought: startCheckout only ever resolves the monthly price id, so a customer selecting yearly and proceeding would have been charged monthly. Fix: the catalog carries the real yearly amounts, the page shows them with the saving computed per tier, and the yearly view states that annual billing is arranged with the team. Wiring self-serve annual checkout is a feature, not a fix, and the price ids for it exist.

Cross-reference: New; same family as A24 and D11 (claims exceeding what ships).

> **FIXED** NEW, found the same way. The pricing page's yearly figures were computed as priceMonthly * 10 with a flat '2 months free' badge on every tier. The real Stripe yearly prices are $299 / $600 / $1,200 — discounts of 14%, 29% and 23% — so the claim was false for all three, and with the corrected monthly prices the formula would have printed '$699.9/yr'. The catalog now carries the real yearly amounts with the saving computed per tier. Annual checkout is still not wired (startCheckout only ever uses the monthly price id), so the yearly view states that annual billing is arranged with the team rather than implying a self-serve path that would charge monthly.

HIGH

A45 The setup docs presented migrations 0001–0004 as the complete set

HIGH

`Evidence: docs/BACKEND.md:50; docs/GO-LIVE.md:35; docs/TESTING.md:14`

Found when applying 0007 failed with “relation "public.google_calendar_connections" does not exist”. All three setup documents listed migrations up to 0004 and called that the full sequence, so a database set up by following them stops before 0005 — which is not a documentation nit: without 0006 the booking flow's post-insert query selects tenants.stripe_charges_enabled and fails, reminders have no booking_reminders table to queue into, and both 0007 and 0009 fail outright. Some of what the earlier audits recorded as broken runtime behaviour may be this gap rather than the code. Fix: the docs list all ten migrations with the cross-gap dependencies stated, and supabase/tests/00_migration_state.sql probes which are genuinely applied — by looking for the objects each one creates, because supabase_migrations.schema_migrations is written only by the CLI and is empty when migrations were pasted into the SQL editor, which is exactly what those docs instructed.

Cross-reference: New; explains why several F-series and D-series runtime observations may not have been code defects at all.

> **FIXED** NEW, found on 19 October when applying 0007 failed with 'relation public.google_calendar_connections does not exist'. BACKEND.md, GO-LIVE.md and TESTING.md all presented migrations 0001-0004 as the complete set, so a database following them stops before 0005 — which leaves the app missing booking_reminders and tenants.stripe_charges_enabled, and makes 0007 and 0009 fail outright. The docs now list all ten with their dependencies, and supabase/tests/00_migration_state.sql probes which are really applied rather than trusting supabase_migrations.schema_migrations, which only the CLI writes.

Items not reproduced, narrowed or corrected

Where the attached Consolidated Master Audit and this review disagree, this is what the source actually shows.

S01–S05 (whole series)

Describes a different repository — Prisma 5, NextAuth 5 beta, Next.js 15.5.9, a FREE tier and a businessId scoping model. None of that exists here: this is Next.js 16.3.1 on Supabase with Postgres RLS and a starter/professional/elite enum. Applying S-series remediation to this codebase would be actively harmful. The only overlapping real issue is the marketing "Enterprise" label (A24).

F16, embed/API instruction half

The dashboard snippet, the onboarding widget reveal and public/embed.js are mutually consistent and the loader is correctly written (same-origin postMessage checks, derives its origin from its own src). Only the marketing Features page shows a non-existent snippet (A25), and the footer's "API docs" link is wrong (A31). The navigation and legal-link part of F16 is confirmed; the "app instructions don't match the loader" part is not.

F03 severity

Confirmed as a defect but downgraded from CRITICAL to HIGH (A08). The endpoint writes only the password already held in the environment and returns no credential, so an unauthorized caller gains no access. The real harms are silent password reversion and an unauthenticated amplification primitive.

F12, plan catalog half

src/lib/plans.ts is already the single typed catalog the audit asks for, with prices, limits and feature flags in one place. The defects are that the marketing page does not read from it (A24) and that nothing enforces it server-side (A19) — not that the catalog is missing.

D06 trial state

Could not be reproduced or refuted from source: trial_ends_at is set at signup and subscription_status is reconciled by the webhook, but the "0 days left while still Trialing" display depends on live data. The related real defect found is A33 — unknown Stripe statuses are mapped to active.

Build and type health

The October 4 QA claim of a clean build and typecheck is confirmed: tsc --noEmit reports 0 errors, next build compiles all 34 routes, eslint reports 0 errors. None of the findings above are compilation problems.

Remediation gates

Four ordered gates. Nothing in a later gate should ship before the gate above it is closed.

Gate 1 — Stop the bleeding (must land before any launch)

- A01 — trigger blocking role/tenant_id self-mutation, with cross-tenant tests
- A02, A04 — move SMTP and OAuth credentials out of tenant-readable rows; rotate anything entered
- A03 — sign or nonce the OAuth state and re-resolve identity from the session
- A05 — fail closed on a missing CRON_SECRET; set it everywhere
- A08 — token-protect or delete /api/admin/setup
- A27 — add security headers
- A37 — guard seed.sql against non-local databases

Gate 2 — Booking correctness

- A09, A10 — one shared duration/buffer calculator used by slot search, quote and insert
- A11 — deduplicate displayed slots without collapsing capacity
- A14 — stop overwriting flat-rate durations
- A13 — transactional upsert onboarding; persist add-ons; never delete referenced rows
- A21 — server-computed readiness before onboarding_complete
- A28, A38 — explicit recurring outcomes; conflict pre-check on reschedule

Gate 3 — Money

- A06 — reconcile Stripe env var names; add a required-variable startup check
- A07 — unique index on payments.stripe_payment_intent_id; reconcile booking state from events
- A12 — complete the payment capture flow; honest paid/unpaid state
- A17 — real invoice and refund actions, or disable the controls
- A19, A33 — enforce plan limits and features server-side; restrict unknown subscription statuses

Gate 4 — Promises the product makes

- A15 — deliver contact submissions
- A18 — schedule and verify reminders
- A16 — persist coupons and apply them, or mark the feature planned
- A22 — real token-based teammate invitations
- A20, A30 — remove fabricated report figures; define every metric
- A23, A24, A25, A31 — real tenant preview; one plan name; accurate snippet; legal links
- A26, A29, A32, A34, A35, A36 — enforce allowed domains and the cancellation window; rate-limit the public flow; capture demand on full days; owner capacity diagnostics; make placeholder mode opt-in

Remediation

What was changed, what is verified, and what you must do before launch.

What was done

All findings are addressed. The original 42: 34 fixed outright, 7 improved with the remainder stated, 1 withdrawn as my own error. Three more (A43–A45) surfaced during the staging preparation and are fixed. Four migrations were added (0007–0010), plus a read-only migration-state probe and three executable verification test files. Nothing was deployed, and no branch other than master was touched.

What is verified, and what is not

Verified here: typecheck (0 errors), production build (all 38 routes compile), lint (0 problems). Every migration was parsed with libpg_query and all 19 PL/pgSQL function bodies compiled through the PL/pgSQL parser. A column catalog built from the migrations confirms that every INSERT and UPDATE column in the SQL exists, and that every table, column and .rpc() the TypeScript references exists — including every RPC declared in types/database.ts. NOT verified: behaviour. No Postgres and no Docker were available in this environment, so not one line of the new SQL has been executed. RLS effects, the booking logic, the Stripe lifecycle and the invitation flow are all unexercised.

Before you launch — in this order

1. Apply 0007–0010 to a STAGING database restored from a production backup, not to production. 2. Set the missing variables: CRON_SECRET, SUPER_ADMIN_SETUP_TOKEN, and the three STRIPE_PRICE_* names the code reads (see A06 — this one needs your decision about monthly versus yearly price ids). The admin console's System health tab lists what is still missing. 3. Rotate any SMTP password that was stored before 0007, and any Google Calendar connection, since both were readable by lower-privileged roles. 4. Run the tests the findings imply, above all: a customer attempting to set their own role; a customer reading tenants.settings; the back-to-back buffer case; a booking with a duration-bearing add-on; two concurrent bookings of one slot; re-running onboarding on a tenant that has bookings; a Stripe test-mode deposit and refund end to end; an invitation accepted from a second account. 5. Only then promote, and watch the reminder queue and the Stripe webhook for the first day.

Honest remaining risk

The security and correctness fixes are the ones I am most confident in: they are small, local and testable. The largest untested surface is the four migrations, because a migration either applies cleanly or does not — run them on staging first and read the output. The largest remaining product gap is online payment capture (A12), which is deliberately unbuilt and now honestly described; launch with pay-later and add capture as its own piece of work. Nothing in this document closes a finding on your behalf: your audit's own rule stands — a finding is CLOSED only with a commit, test evidence, an environment, a date and a reviewer, and the evidence column above gives you the first and the last of those only.

Closure rules

Close a finding only with a commit, a test that would have caught it, the environment it was verified in, a date and a reviewer. The database findings (A01, A02, A04) are read from migration source: confirm the deployed policy, trigger and column-grant state in Supabase before marking them closed, and treat any credential that was reachable as compromised until rotated. Nothing in this document was executed against production.